

Question No:3

Question description

A game consists of n rooms and m teleporters. At the beginning of each day, you start in room 1 and you have to reach room n .

You can use each teleporter at most once during the game. How many days can you play if you choose your routes optimally?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of rooms and teleporters. The rooms are numbered $1, 2, \dots, n$.

After this, there are m lines describing the teleporters. Each line has two integers a and b : there is a teleporter from the room a to room b .

There are no two teleporters whose starting and ending rooms are the same.

Output

First print an integer k : the maximum number of days you can play the game. Then, print k route descriptions according to the example. You can print any valid solution.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 6 7 1 2 1 3 2 6 3 4 3 5 4 6 5 6 EXPECTED OUTPUT 2 4 1 3 5 6 3 1 2 6	INPUT (STDIN) 6 7 1 2 2 3 3 6 4 4 4 5 5 6 5 6 EXPECTED OUTPUT 1 4 1 2 3 6

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 505
5  #define MAXM 2005
6  #define INF 1000000000
7
8  typedef struct {
9      int to;
10     int capacity;
11     int rev;
12 } Edge;
13
14 Edge graph[MAXN][MAXM];
15 int degree[MAXN];
16
17 int level[MAXN];
18 int ptr[MAXN];
19
20 int n, m;
21
22 /* Add directed edge */
23 void addEdge(int u, int v)
24 {
25     graph[u][degree[u]].to = v;
26     graph[u][degree[u]].capacity = 1;
27     graph[u][degree[u]].rev = degree[v];
28
29     graph[v][degree[v]].to = u;
```

```

30     graph[v][degree[v]].capacity = 0;
31     graph[v][degree[v]].rev = degree[u];
32
33     degree[u]++;
34     degree[v]++;
35 }
36
37 /* BFS to build level graph */
38 int bfs(int source, int sink)
39 {
40     int queue[MAXN];
41     int front = 0, rear = 0;
42
43     for (int i = 1; i <= n; i++)
44         level[i] = -1;
45
46     level[source] = 0;
47     queue[rear++] = source;
48
49     while (front < rear)
50     {
51         int u = queue[front++];
52
53         for (int i = 0; i < degree[u]; i++)
54         {
55             Edge *e = &graph[u][i];

```

```

56
57         if (e->capacity > 0 && level[e->to] == -1)
58         {
59             level[e->to] = level[u] + 1;
60             queue[rear++] = e->to;
61         }
62     }
63 }
64
65     return level[sink] != -1;
66 }
67
68 /* DFS to send flow */
69 int dfs(int u, int sink, int flow)
70 {
71     if (u == sink)
72         return flow;
73
74     for (int *i = &ptr[u]; *i < degree[u]; (*i)++)
75     {
76         Edge *e = &graph[u][*i];
77
78         if (e->capacity > 0 &&
79             level[e->to] == level[u] + 1)
80         {
81             int pushed = dfs(e->to, sink,
82                             flow < e->capacity ? flow : e->capacity);
83
84             if (pushed > 0)
85             {
86                 e->capacity -= pushed;
87                 graph[e->to][e->rev].capacity += pushed;
88
89                 return pushed;
90             }
91         }
92     }
93
94     return 0;
95 }
96
97 /* Dinic maximum flow */
98 int maxFlow(int source, int sink)
99 {
100     int flow = 0;
101
102     while (bfs(source, sink))

```

```

103     {
104         for (int i = 1; i <= n; i++)
105             ptr[i] = 0;
106
107         while (1)
108         {
109             int pushed = dfs(source, sink, INF);
110
111             if (pushed == 0)
112                 break;
113
114             flow += pushed;
115         }
116     }
117
118     return flow;
119 }
120
121 /*
122     Find one used path from 1 to n.
123
124     A forward edge originally had capacity 1.
125     After flow is sent, its reverse edge has
126     capacity 1, meaning that edge is currently
127     carrying flow.
128
129     We find those used edges and remove them

```

```

130     while printing the path.
131 */
132 int findPath(int u, int sink, int visited[], int path[], int *pathLen)
133 {
134     if (u == sink)
135     {
136         path[*pathLen++] = u;
137         return 1;
138     }
139
140     visited[u] = 1;
141
142     for (int i = 0; i < degree[u]; i++)
143     {
144         Edge *e = &graph[u][i];
145
146         if (visited[e->to])
147             continue;
148
149         /*
150          Reverse edge capacity > 0 means
151          flow is currently using u -> e->to.
152          */
153         if (e->capacity == 0)
154         {
155             int v = e->to;

```

```

157         /*
158         Check whether the reverse edge
159         has positive capacity.
160         */
161         if (graph[v][e->rev].capacity > 0)
162         {
163             if (findPath(v, sink, visited, path, pathLen))
164             {
165                 path[(*pathLen)++] = u;
166                 return 1;
167             }
168         }
169     }
170 }
171
172     return 0;
173 }
174
175 int main()
176 {
177     scanf("%d %d", &n, &m);
178
179     for (int i = 0; i < m; i++)

```

```
180     {
181         int a, b;
182
183         scanf("%d %d", &a, &b);
184
185         addEdge(a, b);
186     }
187
188     int source = 1;
189     int sink = n;
190
191     /* Find maximum number of edge-disjoint paths */
192     int answer = maxFlow(source, sink);
193
194     printf("%d\n", answer);
195
196     /*
197     |   Extract each route.
198     */
199     for (int day = 0; day < answer; day++)
200     {
201         int visited[MAXN] = {0};
202         int path[MAXN];
203         int pathLen = 0;
204
205         findPath(source, sink, visited, path, &pathLen);
206
207         /*
208         |   findPath stores path backwards:
```

```

209         n ... 3 2 1
210
211         Print it in correct order.
212     */
213     printf("%d", pathLen);
214
215     for (int i = pathLen - 1; i >= 0; i--)
216     {
217         printf(" %d", path[i]);
218     }
219
220     printf("\n");
221
222     /*
223     Remove the flow from this path so
224     the next path uses different teleporters.
225     */
226     for (int i = pathLen - 1; i > 0; i--)
227     {
228         int u = path[i];
229         int v = path[i - 1];
230
231         for (int j = 0; j < degree[u]; j++)
232         {
233             Edge *e = &graph[u][j];
234
235             if (e->to == v && e->capacity == 0)
236             {
237                 /*
238                 Remove one unit of flow.
239                 */
240                 e->capacity = 1;
241                 graph[v][e->rev].capacity = 0;
242                 break;
243             }
244         }
245     }
246 }
247
248 return 0;
249 }

```

Output:

Test Case-1:

```
6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6
2
3 1 2 6
4 1 3 4 6
```

Test Case-2:

```
6 7
1 2
2 3
3 6
4 4
4 5
5 6
5 6
1
4 1 2 3 6
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA Practice>
```